

REPORT

운영체제 프로젝트1(tsh)



과목명	운영체제론
-----	-------

담당교수	오희국 교수님
------	---------

학생이름	지현도
------	-----

학과	컴퓨터학부
----	-------

학번	2021004866
----	------------

제출일	2023/03/26
-----	------------

HANYANG UNIVERSITY

- 본인이작성한함수에대한설명

- 1)표준 입출력 리다이렉션 '<' '>'

- 리다이렉션'<': Strcmp()를 사용하여 명령어가 '<'인 경우 read_only 로 파이프 디스크립터(fd_in)를 연다.

- 리다이렉션'>': 리다이렉션'<'와 유사하고, 반대로 쓰기 위함이므로 코드가 조금 다르다.

```
/*
 * 표준 입출력 리다이렉션 '<' 를 위한 코드
 */
for (int i = 0; argv[i] != NULL; i++) {
    if (!strcmp(argv[i], "<")) {
        fd_in = open(argv[i+1], O_RDONLY);
        if (fd_in == -1) {
            perror("open");
            exit(EXIT_FAILURE);
        }
        argv[i] = NULL;
        i++;
    }
}

/*
 * 표준 입출력 리다이렉션 '>' 를 위한 코드
 */
else if (!strcmp(argv[i], ">")) {
    fd_out = open(argv[i+1], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd_out == -1) {
        perror("open");
        exit(EXIT_FAILURE);
    }
    argv[i] = NULL;
    i++;
}
}
```

리다이렉션 함수 부분이라 리다이렉션 예외처리라고
주석을 달아놨는데 사실 파일디스크립터
예외처리라고도 볼 수 있을 것 같다.

```
// 리다이렉션 예외처리
if (fd_in != STDIN_FILENO) {
    if (dup2(fd_in, STDIN_FILENO) == -1) {
        perror("dup2");
        exit(EXIT_FAILURE);
    }
    close(fd_in);
}

if (fd_out != STDOUT_FILENO) {
    if (dup2(fd_out, STDOUT_FILENO) == -1) {
        perror("dup2");
        exit(EXIT_FAILURE);
    }
    close(fd_out);
}
```

2)파이프

우선 파이프가 몇개 필요할지 모르므로 '|'를 만날때마다
파이프의 개수를 세주었다. 그런후 |의 개수 즉
num_cmds 만큼 파이프디스크립터(pipefds)를 생성해주었다.

이때 2 차원배열로 디스크립터를 생성해주었는데,
pipefds[i][j]에서 [i]는 몇번째 파이프인지, [j]는 입력인지,
출력인지를 나타낸다.(0==fd_in, 1==fd_out)

```
184      /* 다중 파이프를 위한 명령어 파싱. */
185      char *cmds[MAX_LINE/2+1];          /* 파이프를 위한 버퍼 생성 */
186      int num_cmds = 0;                   /* 파이프의 개수를 세기 위한 변수 */
187      cmds[num_cmds++] = cmd;
188
189      p = strchr(cmd, '|');
190      while (p != NULL) {
191          *p = '\0';
192          cmds[num_cmds++] = p + 1;
193          p = strchr(p + 1, '|');
194      }
195
196      int pipefds[num_cmds - 1][2];      /* 파이프 생성 */
197      for (int i = 0; i < num_cmds - 1; i++) {
198          if (pipe(pipefds[i]) == -1) {
199              perror("pipe");
200              exit(EXIT_FAILURE);
201          }
202      }
```

그 후 생성된 파이프 개수를 토대로 각 파이프에 해당할
파이프디스크립터(pipefds)의 값을 변경해주었다.
num_cmds 는 기본 1 시작이다.

```
204      pid_t child_pids[num_cmds];        /* 자식pid 생성(파이프) */
205      for (int i = 0; i < num_cmds; i++) {
206          fd_in = i == 0 ? STDIN_FILENO : pipefds[i-1][0];
207          fd_out = i == num_cmds - 1 ? STDOUT_FILENO : pipefds[i][1];
208
209          if ((child_pids[i] = fork()) == -1) {
210              perror("fork");
211              exit(EXIT_FAILURE);
212          }
```

다음은 자식 프로세스의 실행 과정이다. 즉 child_pids 가
정상적으로 생성되었다면, 파일디스크립터들을

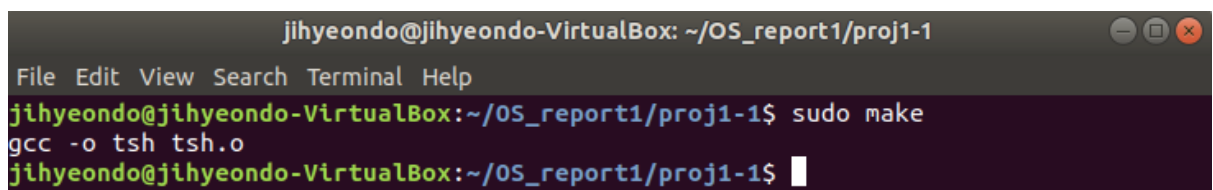
닫아주고(close()) cmdexec 함수로 인자들을 보내준다(cmds[i],
STDIN_FILENO, STDOUT_FILENO).

```
213      /*
214      * 자식 프로세스
215      */
216      else if (child_pids[i] == 0) {
217          if (fd_in != STDIN_FILENO) {
218              if (dup2(fd_in, STDIN_FILENO) == -1) {
219                  perror("dup2");
220                  exit(EXIT_FAILURE);
221              }
222              close(fd_in);
223          }
224
225          if (fd_out != STDOUT_FILENO) {
226              if (dup2(fd_out, STDOUT_FILENO) == -1) {
227                  perror("dup2");
228                  exit(EXIT_FAILURE);
229              }
230              close(fd_out);
231          }
232
233          for (int j = 0; j < num_cmds - 1; j++) {
234              close(pipefds[j][0]);
235              close(pipefds[j][1]);
236          }
237
238          cmdexec(cmds[i], STDIN_FILENO, STDOUT_FILENO);
239          exit(EXIT_SUCCESS);
240      }
241  }
242
243  for (int i = 0; i < num_cmds - 1; i++) {
244      close(pipefds[i][0]);
245      close(pipefds[i][1]);
246  }
```

그후 파이프들이 모두 끝날때까지 기다려준다.

```
247      /*
248          * 포그라운드 실행이면 부모 프로세스는 자식이 끝날 때까지 기다린다.
249          * 백그라운드 실행이면 기다리지 않고 다음 명령어를 입력받기 위해 루프의 처음으로 간다.
250          */
251      if (!background) {
252          for (int i = 0; i < num_cmds; i++) {
253              waitpid(child_pids[i], NULL, 0);
254          }
255      }
256  }
257
258  return 0;
259 }
260
```

- 컴파일과정을보여주는화면캡처 : 오류없이 정상적으로 컴파일 완료가 되는 모습.



```
jihyeondo@jihyeondo-VirtualBox: ~/OS_report1/proj1-1
File Edit View Search Terminal Help
jihyeondo@jihyeondo-VirtualBox:~/OS_report1/proj1-1$ sudo make
gcc -o tsh tsh.o
jihyeondo@jihyeondo-VirtualBox:~/OS_report1/proj1-1$
```

- 실행결과물에대한상세한설명

-검증

1. 기본 명령

1)tsh> grep int tsh.c : 'int'라고 적혀있는 부분을 tsh.c 에서 찾아온다.

```
tsh> grep int tsh.c
static void cmdexec(char *cmd, int fd_in, int fd_out) {
    int argc = 0; /* 인자의 개수 */
    for (int i = 0; argv[i] != NULL; i++) {
int main(void)
    int len; /* 입력된 명령어의 길이 */
    int background; /* 백그라운드 실행 유무 */
    int fd_in, fd_out; // pipe
        printf("[%d] + done\n", pid);
    printf("tsh> "); fflush(stdout);
    int num_cmds = 0;
    int pipefds[num_cmds - 1][2];
    for (int i = 0; i < num_cmds - 1; i++) {
    for (int i = 0; i < num_cmds; i++) {
        for (int j = 0; j < num_cmds - 1; j++) {
    for (int i = 0; i < num_cmds - 1; i++) {
        for (int i = 0; i < num_cmds; i++) {
tsh>
```

2)tsh> grep "if.*NULL" tsh.c & : 마찬가지로 grep 명령어를 이번에는 백그라운드로 실행

```
tsh> grep "if.*NULL" tsh.c &
tsh>         if (q == NULL || *q == ' ' || *q == '\\t') {
            if (p != NULL) {

[5409] + done
tsh>
```

3)tsh> ps : ps 명령어를 실행

```
tsh> ps
  PID TTY          TIME CMD
 2206 pts/0        00:00:00 dbus-launch
 5379 pts/0        00:00:00 sudo
 5380 pts/0        00:00:00 tsh
 5434 pts/0        00:00:00 ps
tsh> █
```

2. 표준 입출력 리다이렉션

1)tsh> grep "int " < tsh.c : 'int' 를 tsh.c 파일에서 읽어온다.

```
tsh> grep "int" < tsh.c
static void cmdexec(char *cmd, int fd_in, int fd_out) {
    int argc = 0;                                /* 인자의 개수 */
    for (int i = 0; argv[i] != NULL; i++) {
int main(void)
    int len;                                    /* 입력된 명령어의 길이 */
    int background;                            /* 백그라운드 실행 유무 */
    int fd_in, fd_out;                        // pipe
        printf("[%d] + done\n", pid);
    printf("tsh> "); fflush(stdout);
    int num_cmds = 0;
    int pipefds[num_cmds - 1][2];
    for (int i = 0; i < num_cmds - 1; i++) {
    for (int i = 0; i < num_cmds; i++) {
        for (int j = 0; j < num_cmds - 1; j++) {
    for (int i = 0; i < num_cmds - 1; i++) {
        for (int i = 0; i < num_cmds; i++) {
tsh> █
```

2)tsh> ls -l : ls -l 명령어를 수행

```
tsh> ls -l
total 40
-rw-r--r-- 1 jihyeondo jihyeondo  408  1월 10 14:43 Makefile
-rwxr-xr-x 1 root      root        13192  3월 25 21:44 tsh
-rw-r--r-- 1 root      root        8513  3월 25 20:56 tsh.c
-rw-r--r-- 1 root      root        6208  3월 25 20:56 tsh.o
tsh> █
```

3)tsh> ls -l > delme : ls -l 명령어의 결과값을 delme 에 저장

```
tsh> ls -l > delme
tsh>
```

4)tsh> cat delme : delme 의 내용물을 cat 명령어로 확인, ls -l 의 결과값이 들어있는 것을 알 수 있다.


```
tsh> cat delme
total 40
-rw-r--r-- 1 root      root          0  3월  25  22:22 delme
-rw-r--r-- 1 jihyeondo jihyeondo    408  1월  10  14:43 Makefile
-rwxr-xr-x 1 root      root     13192  3월  25  21:44 tsh
-rw-r--r-- 1 root      root     8513   3월  25  20:56 tsh.c
-rw-r--r-- 1 root      root     6208   3월  25  20:56 tsh.o
tsh>
```

5)tsh> sort < delme > delme2 : delme 의 내용물을 정렬(오름차순)한 후 delme2 에 저장.

```
tsh> sort < delme > delme2
```

6)tsh> cat delme2 : delme2 의 내용물을 확인

```
tsh> cat delme2
-rw-r--r-- 1 jihyeondo jihyeondo    408  1월  10  14:43 Makefile
-rw-r--r-- 1 root      root          0  3월  25  22:22 delme
-rw-r--r-- 1 root      root     6208   3월  25  20:56 tsh.o
-rw-r--r-- 1 root      root     8513   3월  25  20:56 tsh.c
-rwxr-xr-x 1 root      root     13192  3월  25  21:44 tsh
total 40
tsh>
```

3. 파이프

1)tsh> ps -A | grep -i system : ps -A 의 결과값을 파이프를 사용하여 grep -i system 의 입력값(경로)으로 사용. system 이라는 글자에 해당하는 소스를 보여준다.

```
tsh> ps -A | grep -i system
  1 ?          00:00:12 systemd
281 ?          00:00:01 systemd-journal
288 ?          00:00:00 systemd-udevd
377 ?          00:00:00 systemd-resolve
579 ?          00:00:00 systemd-logind
1094 ?         00:00:00 systemd
1501 ?         00:00:00 systemd
tsh>
```

2)tsh> ps -A | grep -i system | awk '{print \$1,\$4}' :
파이프를 두개 사용하여 ps -A | grep -i system 의
결과값을 awk '{print \$1,\$4}' 의 입력값(경로)으로 사용.
1 번째와 4 번째 행에 해당하는 값들을 출력해준다.

```
tsh> ps -A | grep -i system | awk '{print $1,$4}'
1 systemd
281 systemd-journal
288 systemd-udevd
377 systemd-resolve
579 systemd-logind
1094 systemd
1501 systemd
tsh>
```

3)tsh> cat tsh.c | head -6 | tail -5 | head -1 : 마찬가지로
파이프를 이번엔 세개 사용하여 명령어들을 이어준다.
tsh.c 의 맨앞 6 줄을 가져온후 그 6 줄의 끝부분부터
5 줄, 다시 가져온 것의 맨앞 1 줄을 출력해준다. 여기서
코드의 가장 맨앞에 존재하는 오희국 교수님의 저작권
주석의 두번째 줄을 출력해준다.

```
tsh> cat tsh.c | head -6 | tail -5 | head -1
* Copyright(c) 2023 All rights reserved by Heekuck Oh.
tsh>
```

4. 조합

4)tsh> sort < tsh.c | grep "int " | awk '{print \$1,\$2}' >
delme3 : 앞서 했던 리다이렉션과 파이프를 혼합한
명령어로 tsh.c 를 정렬한 후 'int'를 찾고 그 결과값의
1,2 번째줄만 가져온 후 delme3 에 저장한다.

```
tsh> sort < tsh.c | grep "int" | awk '{print $1,$2}' > delme3
tsh> █
```

```
tsh> cat delme3
for (int
for (int
for (int
for (int
for (int
for (int
int argc
int background;
int fd_in,
int len;
int main(void)
int num_cmds
int pipefds[num_cmds
printf("[%d] +
printf("tsh> ");
static void
tsh> █
```

- 과제를 수행하면서 경험한 문제점과 느낀점

: 우선 처음엔 기본적으로 학습했던 1,2 개의 파이프가 아닌 3 개이상의 다중 파이프를 구현해야한다는 점에서 애로사항이 있었습니다. 결국 프로그램(셸) 목적상 파이프가 수십개 수백개 필요할 수도 있기 때문에 파이프의 개수를 유동적으로 설계할 필요성이 있었습니다. 따라서 2 차원 배열을 활용하여 파이프의 개수를 조절해봤습니다. 지금 당장은 정상적으로 코드가

수행되지만 이렇게 설계하는 것이 맞는지에 대한 피드백이 필요할 것 같습니다.

또한 제 실력이 부족하고 또 시간이 부족한 관계로 cmdexec 함수에 파이프기능을 몰아넣지 못하고 메인함수에서 파이프개수를 확인하고 그걸 cmdexec 로 리턴해주는 방식으로 코드를 작성하였는데, 이부분이 개인적으로 아쉬운 결과물이라고 생각합니다. 메인함수의 내용이 길어지는건 가독성 측면에서 좋지 않다고 인지하고 있기 때문에, 결과물이 이렇다는 것에 아쉬움이 남습니다.

이번 과제로 파이프 즉 부모 프로세스와 자식 프로세스간의 통신과정과 셸의 구성요소를 이해해보는 시간이 된 것 같습니다.